SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE: INTERNET PROGRAMMING

COURSE CODE: CSA4304

COURSE OUTCOME: CO-1

APPLICATION-BASED QUESTIONS

Total Marks: 30 | Weightage: 40%

Student Name	B MAHESH BABU
Register Number	192311189
Department / Class	B. Tech CSE
Academic Year	2023–2027
Submission Mode	Online

Student Declaration

I, **MAHESH BABU**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student	B Mahesh Babu
Date	3-10-2026

.

Assessment Overview

The assessment asks the student to apply HTML5, web standards, HTTP communication, form handling, accessibility, security, and cross-browser compatibility concepts to three practical web-development scenarios.

Question	Application Area	Sub-questions
1	Online Symposium Portal	5
2	Cross-Browser Compatibility Issue	5

3	Form Submission Failure	5

QUESTION 1 – ONLINE SYMPOSIUM PORTAL

Given situation: A college is developing an online symposium registration portal. The supplied form uses POST and contains name and email fields. The assessment asks for semantic HTML, standards compliance, HTTP request/response analysis, and usability/accessibility improvements.

1.1 Identify appropriate HTML5 semantic elements missing in the structure.

Answer: The original form contains the controls needed for data entry, but it does not show the broader semantic page structure. Appropriate HTML5 semantic elements include **<header>** for the page or portal heading, **<main>** for the primary content, **<section>** for the registration area, and **<footer>** for supporting information. The form itself should remain a **<form>** element. Each input should also have a corresponding **<label>** so that the purpose of the control is clear to users and assistive technologies.

1.2 Modify the structure to make it standards-compliant.

A standards-compliant version should declare HTML5, specify the document language and character encoding, include the viewport setting, associate labels with controls, and explicitly identify the submit button.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Symposium Registration</title>
</head>
<body>
    <header>
        <h1>College Symposium Registration</h1>
    </header>

    <main>
        <section aria-labelledby="registration-heading">
            <h2 id="registration-heading">Register for the Symposium</h2>

            <form action="/register" method="POST">
                <div>
                    <label for="name">Full Name:</label>
<input id="name" name="name" type="text"
autocomplete="name" required>                </div>

                <div>
                    <label for="email">Email Address:</label>
<input id="email" name="email" type="email"
autocomplete="email" required>                 </div>

                <button type="submit">Submit Registration</button>
            </form>
        </section>
    </main>

    <footer>
        <p>&copy; College Symposium</p>
    </footer>
</body>
</html>
```

1.3 Interpret the HTTP request generated during form submission.

When the student activates the submit button, the browser validates the form controls and, if validation succeeds, constructs an HTTP **POST** request to the URL represented by **/register**. The form field names become the keys in the request body. A simplified request can be represented as follows:

```
POST /register HTTP/1.1
Host: example.edu
Content-Type: application/x-www-form-urlencoded Content-
Length: ...

name=Mahesh+Babu&email=student%40example.com
```

The important point is that POST sends the submitted form data in the request body rather than appending it to the URL query string. The browser also sends headers such as Host and Content-Type. The server receives the request, parses the submitted fields, validates them, performs the required registration operation, and generates a response.

1.4 Analyze the HTTP response cycle from server to browser.

The response cycle can be understood in the following sequence:

Step	Process
1	Browser sends the POST request to the server endpoint /register.
2	The web server or application receives and parses the request body.
3	Server validates the submitted name and email and performs registration-related processing.
4	Server creates an HTTP response containing a status code, response headers, and optionally a response body.
5	Browser receives the response and interprets it according to the status code and content type.
6	The browser renders the returned HTML or follows a redirect if the server instructs it to do so.

For example, a successful submission may result in a response such as **HTTP/1.1 200 OK** when the server directly returns a page, or **HTTP/1.1 303 See Other** when the server redirects the browser to a result page after processing. The exact status depends on the server implementation.

1.5 Suggest improvements for usability and accessibility.

1. Use a visible **<label>** for every form control and associate it with the correct input using **for** and **id**.
2. Use appropriate input types such as **email** so browsers can provide suitable validation and mobile keyboards.
3. Use **required** for mandatory fields and provide clear, understandable validation messages.
4. Group related fields with **<fieldset>** and **<legend>** if the form becomes larger.
5. Provide keyboard-accessible controls and ensure visible focus indicators.
6. Maintain sufficient color contrast and do not communicate important information through color alone.
7. Use descriptive button text such as **Submit Registration** instead of an ambiguous label such as Submit.

8. Use HTTPS for the registration portal so data is encrypted during transmission.
9. Preserve user-entered values when validation fails, where appropriate, so the user does not need toretype information.

QUESTION 2 – CROSS-BROWSER COMPATIBILITY ISSUE

The supplied snippet contains an incomplete heading and paragraph structure. The assessment asks the student to identify the markup problem, explain browser interpretation, add semantics, correct the document, and suggest compatibility techniques.

2.1 Identify improper nesting and missing closing tags.

The snippet opens **<h1>** and ** ** but does not explicitly close them before the surrounding ** ** is closed. The intended structure should close the heading with **</h1>** and the paragraph with ** **. The ** ** itself is closed, and the document's outer **<html>**, **<head>**, and **<body>** elements are shown as closed.

2.2 Analyze how different browsers may interpret this structure.

HTML browsers use parsing algorithms that attempt to construct a usable document tree even when the source contains errors. Because the supplied markup is incomplete, a browser may implicitly create or close elements while constructing the DOM. Different rendering engines can therefore produce differences in the resulting DOM tree, default spacing, CSS targeting, or layout behavior, particularly when additional styles or scripts depend on the expected structure. Relying on error recovery is unsafe because the developer no longer controls the exact document structure.

2.3 Identify missing semantic elements such as header and section.

The page would be more meaningful if its content were organized using semantic HTML5 elements. A **<header>** can contain the company or page heading, **<main>** can contain the primary page content, **<section>** can group the About Us content, and **<footer>** can contain supporting information. A **<nav>** would be appropriate if the page also contains primary navigation links.

2.4 Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Company</title>
</head>
<body>
    <header>
        <h1>Welcome</h1>
    </header>

    <main>
        <section aria-labelledby="about-heading">
            <h2 id="about-heading">About Us</h2>
            <p>About us</p>
        </section>
    </main>

    <footer>
        <p>&copy; Company</p>
    </footer>
</body>
</html>
```

The corrected structure explicitly closes every element, uses a valid HTML5 document declaration, provides a language attribute, and replaces the generic wrapper with meaningful semantic regions. The heading hierarchy is also improved by using **<h1>** for the page title and **<h2>** for the section title.

2.5 Suggest techniques to ensure cross-browser compatibility.

1. Validate HTML and CSS during development so structural errors are detected early.
2. Use standards-based HTML5 and CSS rather than browser-specific or obsolete markup.
3. Use a consistent CSS reset or normalization strategy when default browser styles need to be controlled.
4. Test the site in the browsers and device sizes relevant to the target users.
5. Use feature detection rather than assuming a feature exists in every browser.
6. Use progressive enhancement: provide a functional basic experience first, then add advanced featureswhere supported.
7. Keep CSS selectors, JavaScript APIs, and HTML semantics compatible with the required browsersupport range.
8. Use responsive design techniques such as flexible layouts and media queries.
9. Retest after changes, especially when CSS or JavaScript modifies the DOM.

QUESTION 3 – FORM SUBMISSION FAILURE

The supplied login form uses method="GET" and a button with type="button". The assessment asks the student to identify the configuration problems, explain why the request is not triggered, interpret GET, propose a correction, and improve secure transmission.

User Input → Login Button → Form Submission → HTTP POST Request → Server → Validate Credentials → Success / Error Response → Browser

3.1 Identify issues in the form configuration.

There are two primary issues:

Issue 1 – Button type: <button type="button"> is a normal button and does not submit its enclosing form by default. Therefore, clicking it does not automatically initiate native form submission.

Issue 2 – GET for credentials: The form uses **method="GET"** while collecting a password. GET encodes submitted form fields into the URL query string, which is unsuitable for credentials because URLs can be exposed through browser history, logs, bookmarks, analytics, proxy records, and other systems.

3.2 Analyze why the HTTP request is not triggered.

The immediate cause is the button declaration. A button with **type="button"** does not perform the form's submit action. Native submission requires **type="submit"**, or a script must explicitly call the form's submission mechanism after performing the required validation. Therefore, simply clicking the supplied Login button does not cause the browser to construct and send the form request.

3.3 Interpret the role of GET method in this scenario.

GET is designed primarily for retrieving a resource. Form data submitted with GET is generally encoded into the target URL as a query string. For example:

```
GET /submit?username=mahesh&password=example123 HTTP/1.1
Host: example.edu
```

This makes GET inappropriate for a password-bearing login form. Even when HTTPS is used, the URL itself can still be recorded by components that handle or log URLs. For a login operation, the credentials should be sent in the request body using POST over HTTPS.

3.4 Propose a corrected implementation.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Secure Login</title>
</head>
<body>
    <main>
        <section aria-labelledby="login-heading">
            <h1 id="login-heading">Login</h1>

            <form action="/submit" method="POST">
                <div>
                    <label for="username">Username:</label>
<input id="username" name="username"
                           type="text" autocomplete="username" required>
</div>
```

```
                <div>
                    <label for="password">Password:</label>
<input id="password" name="password"
                           type="password" autocomplete="current-password"
required>                 </div>

                <button type="submit">Login</button>
            </form>
        </section>
    </main>
</body>
</html>
```

This corrected form uses POST for the login operation and changes the button to **type="submit"**. Labels, required validation, and autocomplete attributes also improve usability and accessibility.

3.5 Suggest improvements for secure data transmission.

1. Use **HTTPS/TLS** for the complete login process so credentials are encrypted in transit.
2. Use **POST** for the login submission rather than placing credentials in a URL.
3. Never store passwords as plain text on the server. Passwords should be processed using a modernpassword-hashing mechanism with an appropriate salt.
4. Validate input on the server even when browser-side validation is present, because client-sidevalidation can be bypassed.
5. Use secure, appropriately configured session cookies after authentication, including protections such asSecure and HttpOnly and suitable SameSite settings.
6. Implement protections against common web attacks such as CSRF where the application'sauthentication architecture requires them.
7. Avoid exposing passwords in URLs, application logs, error messages, or client-side debug output.
8. Apply sensible rate limiting and account-protection controls to reduce automated login abuse.

Overall Technical Analysis

The three scenarios demonstrate that correct web development requires more than writing markup that appears to work. The page structure should be valid and semantic, form controls should have explicit purpose and accessible labels, and the browser's HTTP interaction should be understood at the request/response level.

In Question 1, the registration portal benefits from semantic HTML5 structure, accessible labels, POST submission, validation, and an explicit HTTP request/response understanding. In Question 2, correcting the markup prevents the browser from having to repair invalid or incomplete structure and improves consistency across rendering engines. In Question 3, changing the button to a submit control fixes the immediate form-submission problem, while changing the login operation to POST and using HTTPS addresses the security requirements associated with credential transmission.

Key Takeaways

- **Semantic HTML:** Use elements according to their meaning, such as header, main, section, nav, and footer.

- **Valid structure:** Close elements correctly and use a complete HTML5 document structure.
- **Forms:** Use labels, suitable input types, required validation, and an explicit submit button.
- **HTTP:** Understand that a request contains a method, target, headers, and potentially a body; the response contains a status, headers, and potentially a body.
- **GET vs POST:** GET places form data in the URL, while POST sends the form data in the request body.
- **Security:** Password-bearing authentication forms should use POST over HTTPS and should not expose credentials through URLs or logs.
- **Accessibility:** Clear labels, keyboard access, meaningful headings, focus visibility, and understandable validation improve usability for a broad range of users.